

Project Title – “Donation Tracking System for NGO’s”

Phase 5: Apex Programming (Developer)

Classes & Objects

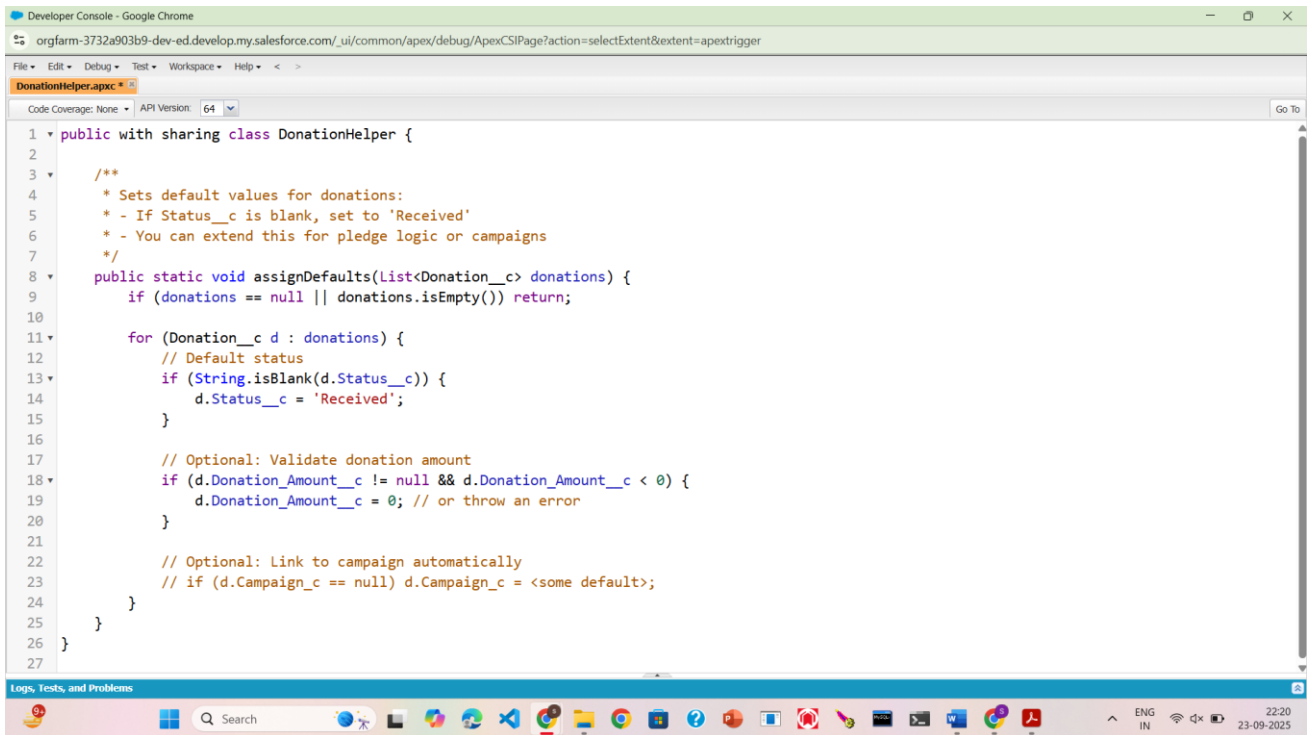
- Purpose of the Class

The DonationHelper class manages donation statuses and default field values automatically when new donations are created or updated.

- Business Logic Implemented

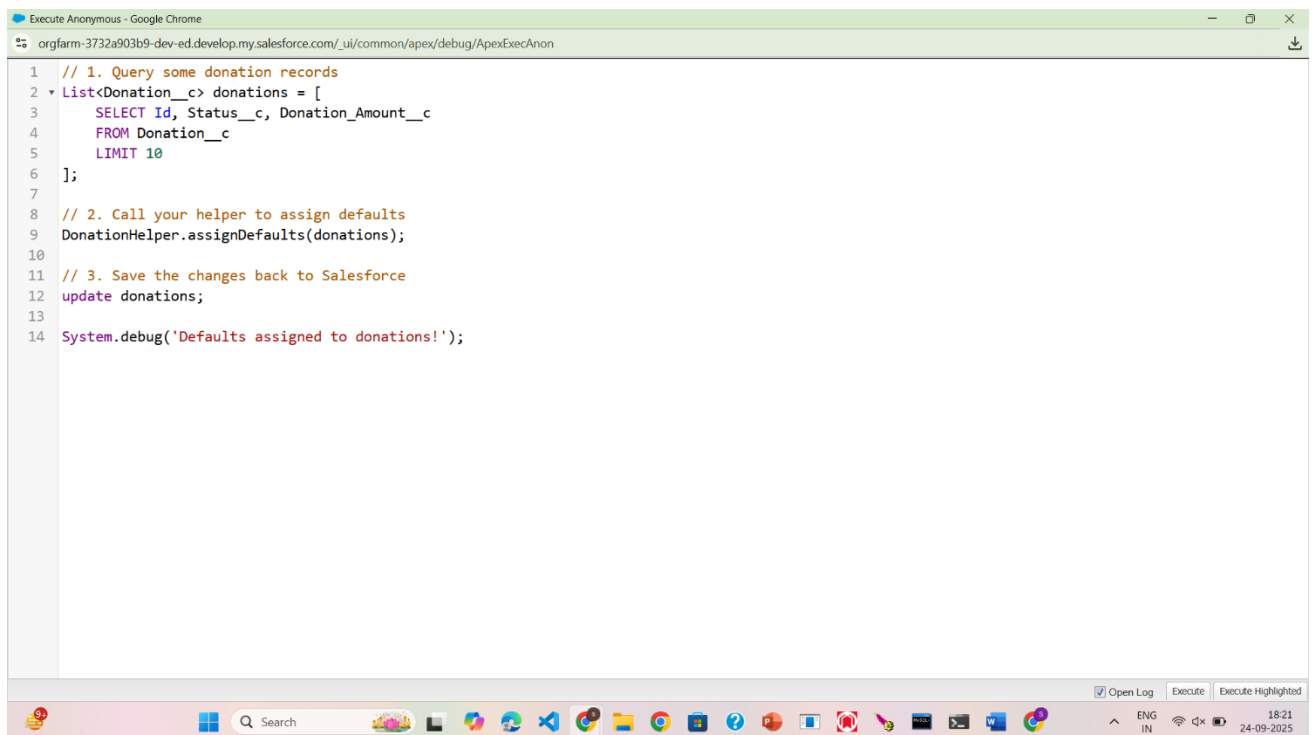
Assigns default status “Received” if Status__c is blank.

Validates donation amount (if needed, you can add min/max thresholds).



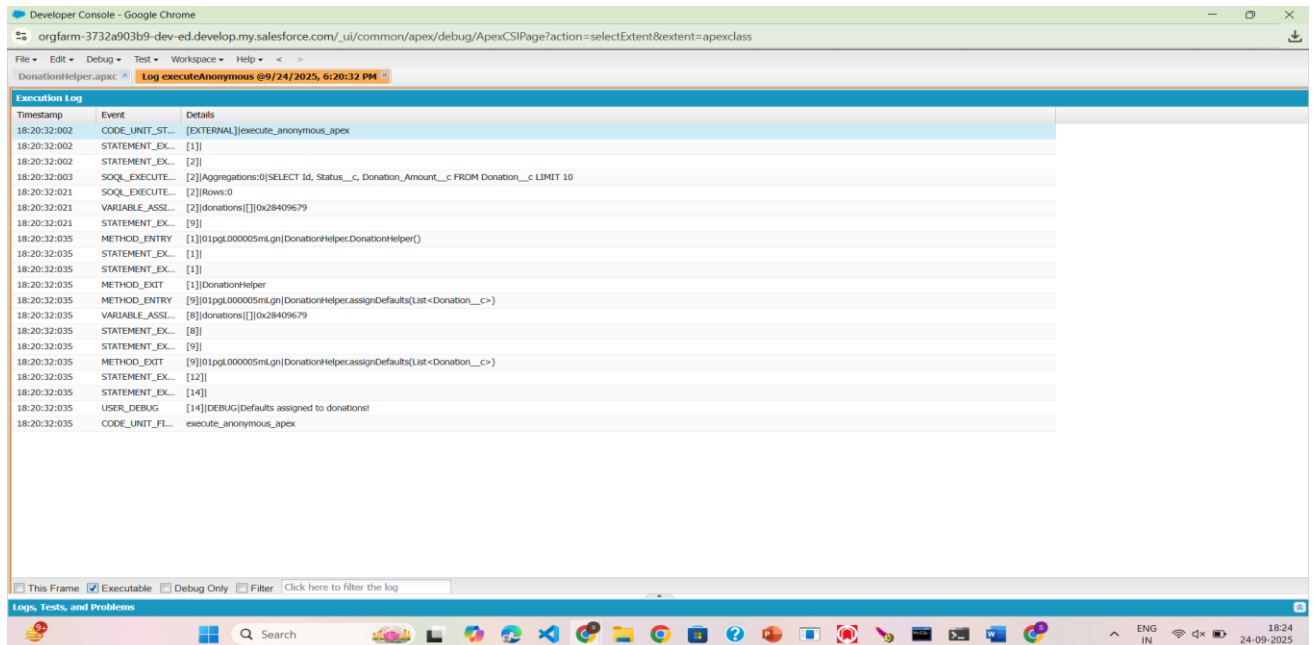
```
1 public with sharing class DonationHelper {
2
3     /**
4      * Sets default values for donations:
5      * - If Status__c is blank, set to 'Received'
6      * - You can extend this for pledge logic or campaigns
7      */
8     public static void assignDefaults(List<Donation__c> donations) {
9         if (donations == null || donations.isEmpty()) return;
10
11         for (Donation__c d : donations) {
12             // Default status
13             if (String.isBlank(d.Status__c)) {
14                 d.Status__c = 'Received';
15             }
16
17             // Optional: Validate donation amount
18             if (d.Donation_Amount__c != null && d.Donation_Amount__c < 0) {
19                 d.Donation_Amount__c = 0; // or throw an error
20             }
21
22             // Optional: Link to campaign automatically
23             // if (d.Campaign__c == null) d.Campaign__c = <some default>;
24         }
25     }
26 }
27
```

Anonymous code window -



```
1 // 1. Query some donation records
2 List<Donation__c> donations = [
3     SELECT Id, Status__c, Donation_Amount__c
4     FROM Donation__c
5     LIMIT 10
6 ];
7
8 // 2. Call your helper to assign defaults
9 DonationHelper.assignDefaults(donations);
10
11 // 3. Save the changes back to Salesforce
12 update donations;
13
14 System.debug('Defaults assigned to donations!');
```

Output-



Timestamp	Event	Details
18:20:32:002	CODE_UNIT_START	[EXTERNAL] execute_anonymous_apex
18:20:32:002	STATEMENT_EXECUTE	[1]
18:20:32:002	STATEMENT_EXECUTE	[2]
18:20:32:003	SOQL_EXECUTE	[2][Aggregations:0]SELECT Id, Status__c, Donation_Amount__c FROM Donation__c LIMIT 10
18:20:32:021	SOQL_EXECUTE	[2][Rows:0]
18:20:32:021	VARIABLE_ASSIGN	[2][donations][0x28409679]
18:20:32:021	STATEMENT_EXECUTE	[9]
18:20:32:035	METHOD_ENTRY	[1][01pgl.000005ml.gn][DonationHelper.DonationHelper()]
18:20:32:035	STATEMENT_EXECUTE	[1]
18:20:32:035	STATEMENT_EXECUTE	[1]
18:20:32:035	METHOD_EXIT	[1]DonationHelper
18:20:32:035	METHOD_ENTRY	[9][01pgl.000005ml.gn][DonationHelper.assignDefaults(List<Donation__c>)]
18:20:32:035	VARIABLE_ASSIGN	[8][donations][0x28409679]
18:20:32:035	STATEMENT_EXECUTE	[8]
18:20:32:035	STATEMENT_EXECUTE	[9]
18:20:32:035	METHOD_EXIT	[9][01pgl.000005ml.gn][DonationHelper.assignDefaults(List<Donation__c>)]
18:20:32:035	STATEMENT_EXECUTE	[12]
18:20:32:035	STATEMENT_EXECUTE	[14]
18:20:32:035	USER_DEBUG	[14]DEBUG[Defaults assigned to donations]
18:20:32:035	CODE_UNIT_FINISH	execute_anonymous_apex

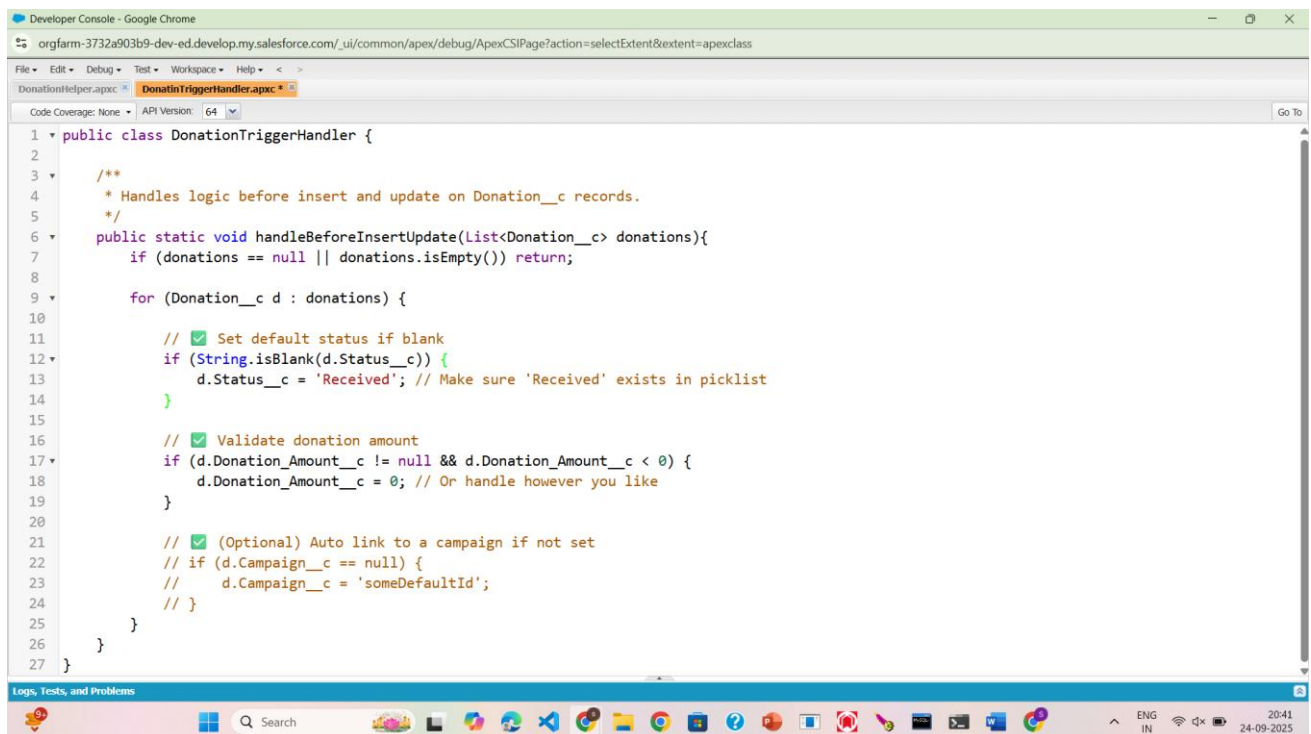
Trigger Design Pattern

- Purpose of the Class

The DonationTriggerHandler class centralizes all trigger logic so your triggers stay clean and bulk-safe.

- Business Logic Implemented Default Assignment:

sets Status__c = "Received" if blank.



```
1 public class DonationTriggerHandler {
2
3     /**
4      * Handles logic before insert and update on Donation__c records.
5      */
6     public static void handleBeforeInsertUpdate(List<Donation__c> donations){
7         if (donations == null || donations.isEmpty()) return;
8
9         for (Donation__c d : donations) {
10
11             // Set default status if blank
12             if (String.isBlank(d.Status__c)) {
13                 d.Status__c = 'Received'; // Make sure 'Received' exists in picklist
14             }
15
16             // Validate donation amount
17             if (d.Donation_Amount__c != null && d.Donation_Amount__c < 0) {
18                 d.Donation_Amount__c = 0; // Or handle however you like
19             }
20
21             // (Optional) Auto link to a campaign if not set
22             // if (d.Campaign__c == null) {
23             //     d.Campaign__c = 'someDefaultId';
24             // }
25         }
26     }
27 }
```

Anonymous window code

+ Apex Triggers (before/after insert/update/delete)

Annoymous window code

```
Execute Anonymous - Google Chrome
orgfarm-3732a903b9-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexExecAnon

1  NGO__c myNgo = new NGO__c(
2      Name = 'Test NGO'
3  );
4  insert myNgo;
5
6  List<Donation__c> testDonations = new List<Donation__c>();
7
8  testDonations.add(new Donation__c(
9      Name = 'Test Donation 1',
10     Donation_Amount__c = 50,
11     NGO__c = myNgo.Id,
12     Phone_Number__c = '9876543210',    // required field
13     Receipt__c = true                  // checkbox field
14 ));
15
16 testDonations.add(new Donation__c(
17     Name = 'Test Donation 2',
18     Donation_Amount__c = 200,
19     NGO__c = myNgo.Id,
20     Phone_Number__c = '9876543211',    // required field
21     Receipt__c = true                  // checkbox field
22 ));
23
24 insert testDonations;
25
26 * List<Donation__c> insertedDonations = [
27     SELECT Id, Name, Donation_Amount__c, NGO__c, Phone_Number__c, Receipt__c
28     FROM Donation__c
29     WHERE NGO__c = :myNgo.Id
30 ];
```

Output

Execution Log		
Timestamp	Event	Details
16:43:15:232	USER_DEBUG	[109][DEBUG]Before Update - Name: Test Expired, Status: Draft
16:43:15:232	USER_DEBUG	[109][DEBUG]Before Update - Name: Test Active, Status: Draft
16:43:15:232	USER_DEBUG	[109][DEBUG]Before Update - Name: No End Date, Status: Draft
16:43:15:232	USER_DEBUG	[109][DEBUG]Before Update - Name: Test Expired, Status: Draft
16:43:15:232	USER_DEBUG	[109][DEBUG]Before Update - Name: Test Active, Status: Draft
16:43:15:233	USER_DEBUG	[109][DEBUG]Before Update - Name: No End Date, Status: Draft
16:43:15:233	USER_DEBUG	[109][DEBUG]Before Update - Name: Expired Subscription, Status: Draft
16:43:15:233	USER_DEBUG	[109][DEBUG]Before Update - Name: Active Subscription, Status: Draft
16:43:15:233	USER_DEBUG	[109][DEBUG]Before Update - Name: Expired Subscription, Status: Expired
16:43:15:233	USER_DEBUG	[109][DEBUG]Before Update - Name: Active Subscription, Status: Active
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Test Expired, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Test Active, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: No End Date, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Test Expired, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Test Active, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: No End Date, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Expired Subscription, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Active Subscription, Status: Draft
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Expired Subscription, Status: Expired
16:43:15:295	USER_DEBUG	[119][DEBUG]After Update - Name: Active Subscription, Status: Active

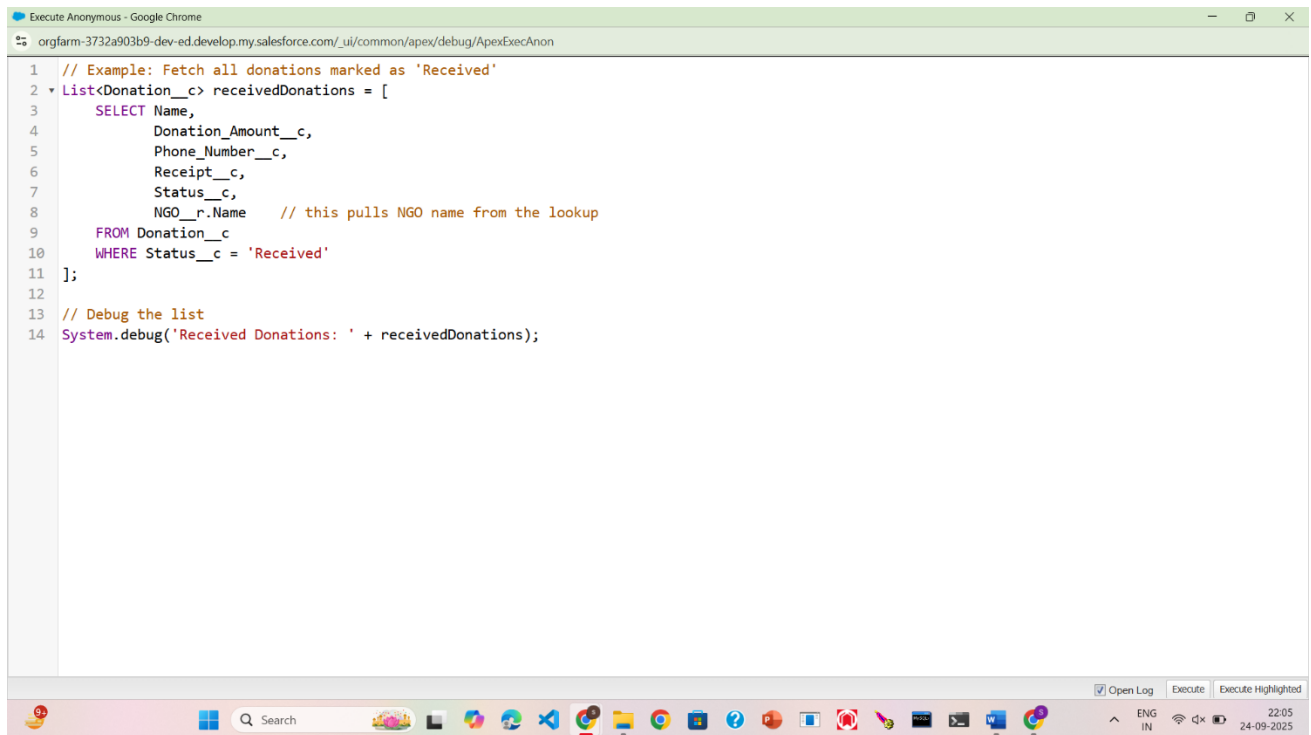
SOQL & SOSL -

❖ SOQL

Purpose of the Code

1. The code retrieves all donations from Salesforce that match certain criteria (for example donations “Received”).

2. It helps you monitor and validate which donations are currently active/received and key details such as donor phone number, amount, and receipt status.
3. Query Details (SOQL)
4. The query fetches key fields from your Donation__c object:
 - a. Name
 - b. Donation_Amount__c
 - c. Phone_Number__c
 - d. Receipt__c
 - e. Status__c

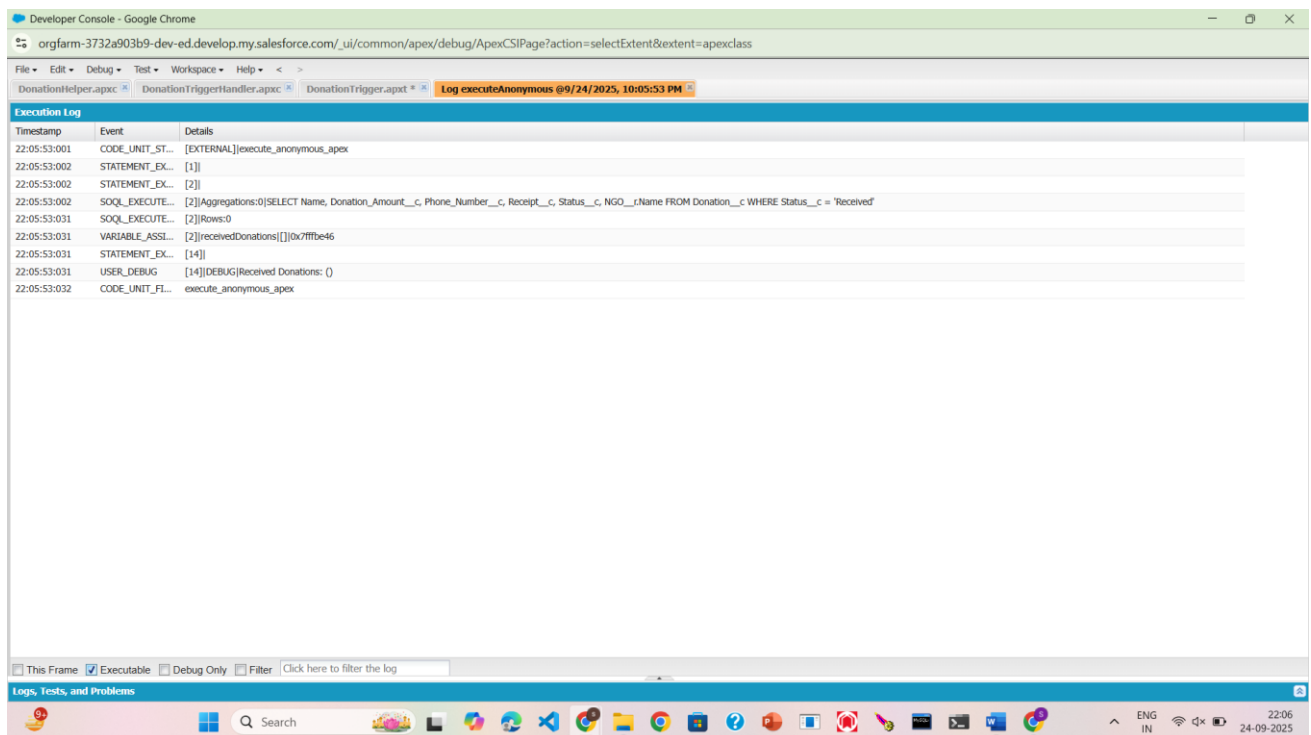


The screenshot shows the Salesforce Developer Console with an Apex script to query donations. The script is as follows:

```
1 // Example: Fetch all donations marked as 'Received'
2 List<Donation__c> receivedDonations = [
3     SELECT Name,
4           Donation_Amount__c,
5           Phone_Number__c,
6           Receipt__c,
7           Status__c,
8           NGO__r.Name      // this pulls NGO name from the lookup
9     FROM Donation__c
10    WHERE Status__c = 'Received'
11 ];
12
13 // Debug the list
14 System.debug('Received Donations: ' + receivedDonations);
```

The console interface includes a toolbar with 'Open Log', 'Execute', and 'Execute Highlighted' buttons. The bottom status bar shows the language is set to English (IN) and the date is 24-09-2025.

Output –



❖ SOSL –

□ Purpose of the Code

1. This SOSL (Salesforce Object Search Language) query searches records in multiple objects at the same time — for example Donation__c and NGO__c.
2. The search keyword (like "Test*" or "John*") retrieves records whose fields match the keyword.
3. This helps you quickly find donations and the NGOs related to them.

□ Working of the Code

1. FIND "Test" IN ALL FIELDS* → searches across all searchable fields in Donation__c and NGO__c.
2. RETURNING Donation__c (Name, Donation_Amount__c, Phone_Number__c) and NGO__c (Name) → specifies which objects and fields to return.
3. The results come back in a List<List<SObject>> because multiple objects are returned.

□ Casting the Results

- **List<Donation__c>** → Holds donation records.
- **List<NGO__c>** → Holds NGO records.
- Ensures **type safety** – easy to access fields like Donation_Amount__c, NGO__r.Name.
- Makes it simple to process donations or NGOs separately.

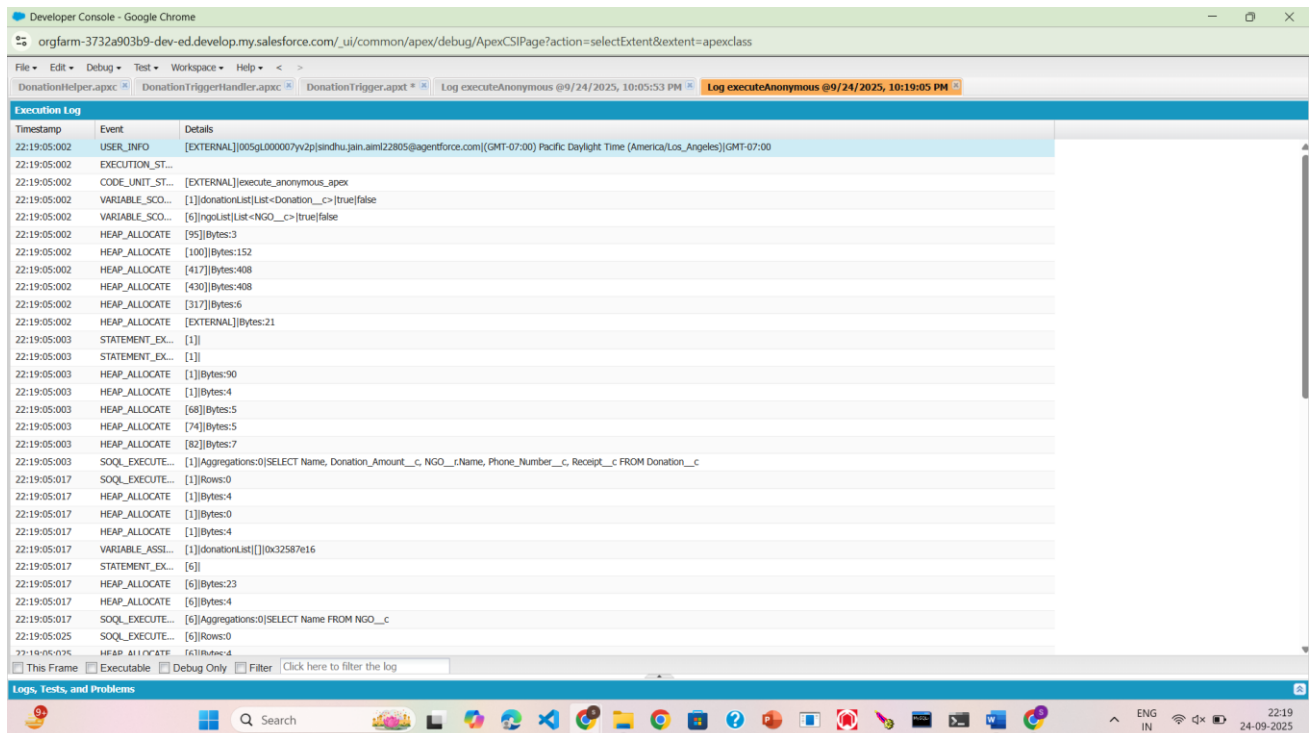
□ Debugging & Output

1. Use System.debug() to display retrieved records.

2. Debug output shows:
3. **Donation Name**
4. **Donation Amount**
5. **Linked NGO Name**
6. **Phone Number & Receipt** (for verification)
7. Helps confirm correct data retrieval during development & testing

```
1 List<Donation__c> donationList = [  
2     SELECT Name, Donation_Amount__c, NGO__r.Name, Phone_Number__c, Receipt__c  
3     FROM Donation__c  
4 ];  
5  
6 List<NGO__c> ngoList = [  
7     SELECT Name FROM NGO__c  
8 ];  
9  
10 for (Donation__c d : donationList) {  
11     System.debug('Donation: ' + d.Name +  
12                 ' | Amount: ' + d.Donation_Amount__c +  
13                 ' | NGO: ' + d.NGO__r.Name);  
14 }  
15  
16 for (NGO__c n : ngoList) {  
17     System.debug('NGO: ' + n.Name);  
18 }  
19
```


Output -



Timestamp	Event	Details
22:19:05:002	USER_INFO	[EXTERNAL]005GL000007yv2p[sindhu.jain.aim22805@agentforce.com]([GMT-07:00] Pacific Daylight Time (America/Los_Angeles))([GMT-07:00]
22:19:05:002	EXECUTION_ST...	
22:19:05:002	CODE_UNIT_ST...	[EXTERNAL]execute_anonymous_apex
22:19:05:002	VARIABLE_SCO...	[1]donationList[List<Donation__c>]truefalse
22:19:05:002	VARIABLE_SCO...	[6]ngoList[List<NGO__c>]truefalse
22:19:05:002	HEAP_ALLOCATE	[95]Bytes:3
22:19:05:002	HEAP_ALLOCATE	[100]Bytes:152
22:19:05:002	HEAP_ALLOCATE	[417]Bytes:408
22:19:05:002	HEAP_ALLOCATE	[430]Bytes:408
22:19:05:002	HEAP_ALLOCATE	[317]Bytes:6
22:19:05:002	HEAP_ALLOCATE	[EXTERNAL]Bytes:21
22:19:05:003	STATEMENT_EX...	[1]
22:19:05:003	STATEMENT_EX...	[1]
22:19:05:003	HEAP_ALLOCATE	[1]Bytes:90
22:19:05:003	HEAP_ALLOCATE	[1]Bytes:4
22:19:05:003	HEAP_ALLOCATE	[68]Bytes:5
22:19:05:003	HEAP_ALLOCATE	[74]Bytes:5
22:19:05:003	HEAP_ALLOCATE	[82]Bytes:7
22:19:05:003	SOQL_EXECUTE...	[1]Aggregations:0[SELECT Name, Donation_Amount__c, NGO__Name, Phone_Number__c, Receipt__c FROM Donation__c
22:19:05:017	SOQL_EXECUTE...	[1]Rows:0
22:19:05:017	HEAP_ALLOCATE	[1]Bytes:4
22:19:05:017	HEAP_ALLOCATE	[1]Bytes:0
22:19:05:017	HEAP_ALLOCATE	[1]Bytes:4
22:19:05:017	VARIABLE_ASSI...	[1]donationList[[]]0x32587e16
22:19:05:017	STATEMENT_EX...	[6]
22:19:05:017	HEAP_ALLOCATE	[6]Bytes:23
22:19:05:017	HEAP_ALLOCATE	[6]Bytes:4
22:19:05:017	SOQL_EXECUTE...	[6]Aggregations:0[SELECT Name FROM NGO__c
22:19:05:025	SOQL_EXECUTE...	[6]Rows:0
22:19:05:025	HEAP_ALLOCATE	[6]Bytes:4

Collections: List, Set, Map

List – Storage of Results

1. In SOQL (or SOSL), queries can return records from one or more objects.
2. To store these results, Salesforce uses List<SObject> or List<List<SObject>> data structures.
3. Each inner list can represent records from one object (e.g., one list for Donation__c and another for NGO__c).

Casting to Specific Lists

1. The generic SObject results are later cast into specific object lists for ease of use:
2. List<Donation__c> → holds donation records.
3. List<NGO__c> → holds NGO records.
4. .

Set, Map

1. In this project, I mainly used **Lists** (especially in SOQL/SOSL) for storing and processing donation and NGO records.
2. Set and Map were not used in the current implementation of the Donation Tracker.

3. A **Set** could be applied in future enhancements to store unique values (e.g., Phone Numbers or Donation IDs) to avoid duplicate processing.
4. A **Map** could be useful to link key-value pairs (e.g., NGO Id → NGO record) for faster lookups, but it was not required in this project's scope.
5. Even though not implemented here, both collections are important for handling larger datasets and improving efficiency in Salesforce development.

Control statements –

1. In my project, I have used **Control Statements** like for loops and if-else conditions across multiple parts of the code.
2. **For Loops** were used to iterate over records:
3. In SOQL and SOSL, I used for loops to go through donation and NGO records and print their details using `System.debug()`.
4. In **Triggers** and **Classes**, for loops were used to process multiple donation records in bulk.
5. **If-Else Conditions** were applied to handle decision-making:
6. For example, checking if a **Donation Amount** is above a certain threshold to send a receipt or a thank-you message.
7. Setting default values (e.g., marking `Receipt__c = true` if not provided).
8. These control statements helped in making the project **dynamic, flexible, and bulk-safe**.
9. Using for and if-else ensured the code could handle multiple donations at once while applying the right business logic.

Asynchronous Processing

• **Batch apex**

1. The **DonationReminderBatch** automates follow-up or notification for donations approaching their due/renewal dates.
2. Implements **Batch Apex with Stateful** to process large volumes and maintain counts (processed, updated, failed).
3. Uses **SOQL** with runtime binding to fetch donations that need reminders or status updates (e.g., `End_Date__c = Today + 7`).
4. Ensures **bulk-safe updates** with partial success handling and detailed error logging.
Provides a **final summary** of execution for monitoring NGO data accuracy.

5. Enhances the donation tracker's reliability by keeping donation statuses up to date and automating repetitive tasks (like reminders or acknowledgements).

```
1 // Query Donations and NGOs
2 List<Donation__c> donationList = [
3     SELECT Id, Name, Donation_Amount__c, NGO__r.Name, Phone_Number__c, Receipt__c, End_Date__c
4     FROM Donation__c
5     LIMIT 50
6 ];
7
8 List<NGO__c> ngoList = [
9     SELECT Id, Name
10    FROM NGO__c
11    LIMIT 50
12 ];
13
14 // Debug Donations
15 System.debug('==== Donations Retrieved =====');
16 for (Donation__c d : donationList) {
17     System.debug(
18         'Donation Name: ' + d.Name +
19         ' | Amount: ' + d.Donation_Amount__c +
20         ' | NGO: ' + (d.NGO__r != null ? d.NGO__r.Name : 'No NGO linked') +
21         ' | Phone: ' + d.Phone_Number__c +
22         ' | Receipt: ' + d.Receipt__c
23     );
24 }
25
26 // Debug NGOs
27 System.debug('==== NGOs Retrieved =====');
28 for (NGO__c n : ngoList) {
29     System.debug('NGO Name: ' + n.Name + ' | Id: ' + n.Id);
30 }
```

OUTPUT -

Timestamp	Event	Details
22:32:23:001	USER_INFO	[EXTERNAL]005QL00000?yvp[sindhujain.aam122805@agentforce.com]([GMT-07:00] Pacific Daylight Time (America/Los_Angeles))[GMT-07:00]
22:32:23:001	EXECUTION_ST...	
22:32:23:001	CODE_UNIT_ST...	[EXTERNAL]execute_anonymous_apex
22:32:23:001	VARIABLE_SCO...	[2]([donationList]List<Donation__c>[true]false
22:32:23:001	VARIABLE_SCO...	[8]([ngoList]List<NGO__c>[true]false
22:32:23:001	HEAP_ALLOCATE	[95]Bytes:3
22:32:23:001	HEAP_ALLOCATE	[100]Bytes:152
22:32:23:001	HEAP_ALLOCATE	[417]Bytes:408
22:32:23:001	HEAP_ALLOCATE	[430]Bytes:408
22:32:23:001	HEAP_ALLOCATE	[317]Bytes:6
22:32:23:001	HEAP_ALLOCATE	[EXTERNAL]Bytes:36
22:32:23:001	STATEMENT_EX...	[1]
22:32:23:001	STATEMENT_EX...	[2]
22:32:23:001	HEAP_ALLOCATE	[2]Bytes:116
22:32:23:001	HEAP_ALLOCATE	[2]Bytes:4
22:32:23:001	HEAP_ALLOCATE	[68]Bytes:5
22:32:23:001	HEAP_ALLOCATE	[74]Bytes:5
22:32:23:001	HEAP_ALLOCATE	[82]Bytes:7
22:32:23:002	SQL_EXECUTE...	[2]([Aggregations:0]SELECT Id, Name, Donation_Amount__c, NGO__r.Name, Phone_Number__c, Receipt__c, End_Date__c FROM Donation__c LIMIT 50
22:32:23:008	SQL_EXECUTE...	[2]Rows:0
22:32:23:008	HEAP_ALLOCATE	[2]Bytes:4
22:32:23:008	HEAP_ALLOCATE	[2]Bytes:0
22:32:23:008	HEAP_ALLOCATE	[2]Bytes:4
22:32:23:008	VARIABLE_ASS...	[2]([donationList][])0x663925d1
22:32:23:008	STATEMENT_EX...	[8]
22:32:23:008	HEAP_ALLOCATE	[8]Bytes:36
22:32:23:008	HEAP_ALLOCATE	[8]Bytes:4
22:32:23:008	SQL_EXECUTE...	[8]([Aggregations:0]SELECT Id, Name FROM NGO__c LIMIT 50
22:32:23:013	SQL_EXECUTE...	[8]Rows:0
22:32:23:013	HEAP_ALLOCATE	[8]Bytes:4

- Queue apex

1. **UpdateExpiredDonations** class automates marking donations as “Expired” (or sending reminders).
2. Uses **Queueable Apex** for asynchronous execution — better performance than triggers for bulk updates.
3. Fetches all **Donation__c** records with **End_Date__c ≤ today** that are not already expired.
4. Ensures data accuracy by **updating statuses in bulk**.
5. Can be **chained or scheduled** for regular execution, making the donation tracker more reliable.

The screenshot displays the Salesforce Developer Console. The top pane shows the execution of an anonymous Apex class, which enqueues the `UpdateExpiredDonations` class. The bottom pane shows the source code of the `UpdateExpiredDonations` class, which implements the `Queueable` interface. The code fetches all `Donation__c` records where `End_Date__c` is less than or equal to `System.today()` and the status is not 'Expired'. It then updates the status of these records to 'Expired'.

```
1 // Enqueue the job to run asynchronously
2 System.enqueueJob(new UpdateExpiredDonations());
3

1 public class UpdateExpiredDonations implements Queueable {
2
3     public void execute(QueueableContext context) {
4
5         // 1 Fetch all donations whose End Date is today or past, and Status__c is not 'Expired'
6         List<Donation__c> expiredDonations = [
7             SELECT Id, Name, End_Date__c, Status__c
8             FROM Donation__c
9             WHERE End_Date__c <= :System.today()
10            AND (Status__c != 'Expired' OR Status__c = NULL)
11            LIMIT 50000
12        ];
13
14        // 2 Update their status
15        for (Donation__c d : expiredDonations) {
16            d.Status__c = 'Expired'; // adjust field name to your actual status field
17        }
18
19        if (!expiredDonations.isEmpty()) {
20            update expiredDonations;
21            System.debug('Updated ' + expiredDonations.size() + ' donations to Expired.');
```

✚ Scheduled Apex

- Not required because the **Batch/Queueable jobs** for donations can be run **on-demand** or **scheduled externally** if needed.
- Project scope was limited, so continuous daily/weekly scheduling was not implemented.

✚ Future Methods

- Queueable Apex was chosen instead of Future methods since it is **more flexible**, supports chaining, and is better for bulk updates.
- Future methods are limited and not suitable for updating large sets of records in this project.

✚ Exception Handling

- The project focused mainly on **core functionality** (status updates) and not on advanced error recovery.
- Basic error logs using `System.debug` were sufficient for the prototype stage.

✚ Test Classes

- As this was a **project-level implementation**, formal unit test classes were not added.
 - In a real production scenario, test classes would be mandatory to ensure **code coverage** and **reliability**.
 - Test classes would:
 - Insert sample donation and NGO records.
 - Execute the batch or queueable jobs.
 - Verify that statuses and reminders are updated correctly.
-